

PROJECT REPORT / SEPTEMBER 16, 2026

The Searchin' Pac-Man

Search implementation, measurement, and critical assessment.

Team member	Stony Brook ID
Maitreya Vaghulade	117437999
Karan Jain	117420953

Principal result

On **trickySearch**, A* finds the same optimal **60-step** route as UCS while expanding **255 versus 16,688 states**. The measured reduction is **98.5%**; median planning time falls from **456.943 ms** to **9.188 ms**.

1. Scope and implementation

Question	Delivered implementation
Q1-Q2	Graph DFS and BFS with duplicate detection and shared parent links.
Q3-Q4	UCS and A* with best-cost tracking, stale-entry rejection, and reopening.
Q5	CornersProblem with immutable position and four-bit visit history.
Q6	Minimum Manhattan tour through unvisited corners; cached suffix costs.
Q7	Nearest-food maze distance plus a minimum spanning tree of remaining food.

Questions 1-7 are implemented in **search.py** and **searchAgents.py**.

2. Measurement protocol

Environment. CPython 3.11.16; macOS 26.6.2; Apple arm64. All measurements were collected locally on September 15, 2026, using the supplied layouts. There are 30 benchmark cases.

Time. Median of five independent, untraced search calls using a monotonic high-resolution clock. Every call receives a new problem with cold heuristic caches. Imports, layout parsing, problem construction, path validation, animation, and game execution are excluded. Heuristic preprocessing and cache construction are included.

Memory. Peak Python allocations measured by `tracemalloc` during one separate cold search per case, including heuristic caches. KiB = 1,024 bytes. This is incremental traced memory, not whole-process resident memory.

Expanded states. The problem's `_expanded` counter counts successor-generation calls. A goal popped from the frontier is not expanded. Heuristic distance preprocessing is included in time and memory, but is not a food-state expansion. Raw JSON preserves every timing sample; tiny timing differences should not be overinterpreted.

Q1 AND Q2

3. Depth-first and breadth-first search

DFS uses the supplied stack and marks a state expanded when it is removed. Successors are pushed in the provided North, South, East, West order, so the last available successor is explored first. **BFS** uses the supplied queue and marks states discovered on insertion, preventing duplicate frontier entries.

Table 1. Unit-cost position search. Steps equal path cost. A* uses Manhattan distance.

Layout	Search	Steps / cost	Expanded	Time (ms)	Peak (KiB)
tinyMaze	DFS	10	15	0.020	4.3
tinyMaze	BFS	8	15	0.018	4.2
tinyMaze	UCS	8	15	0.020	4.4
tinyMaze	A*	8	14	0.020	4.4
mediumMaze	DFS	130	146	0.138	33.0
mediumMaze	BFS	68	269	0.249	45.9
mediumMaze	UCS	68	269	0.295	47.2
mediumMaze	A*	68	221	0.248	47.9
bigMaze	DFS	210	390	0.369	99.4
bigMaze	BFS	210	620	0.608	107.5
bigMaze	UCS	210	620	0.687	93.8
bigMaze	A*	210	549	0.628	89.4
openMaze	DFS	298	576	0.644	149.7
openMaze	BFS	54	682	0.695	110.5
openMaze	UCS	54	682	0.843	133.1
openMaze	A*	54	535	0.679	87.8

Interpretation

On **mediumMaze**, DFS finds a 130-step route after 146 expansions; BFS finds the optimal 68-step route after 269 expansions. DFS commits to a branch and does not compare solution lengths. Its lower expansion count therefore buys speed at the expense of path quality. The assignment gives an approximate DFS length of 245 moves. With the supplied maze and successor order unchanged, this implementation returns a valid 130-move route. Executing the returned moves confirmed that Pac-Man reaches the goal without crossing walls.

The red exploration overlay represents planning, not physical travel. Pac-Man executes only the reconstructed solution; it need not visit explored dead ends. Graph DFS avoids re-expanding visited states and is complete on these finite graphs. BFS is optimal for equal step costs, but not for arbitrary weighted paths. DFS happens to match the optimum on bigMaze; that single result does not give it an optimality guarantee.

Q3 AND Q4

4. Cost-sensitive and informed search

UCS removes the frontier node with the smallest accumulated cost **g**. **A*** removes the node with the smallest **g + h**. Both keep the best known g-value for each state, insert an entry only when that value improves, and skip obsolete entries. Testing the goal on removal prevents a costly goal generated early from ending the search.

Equal priorities use insertion order from the supplied priority queue. Improved routes can reopen a state, supporting admissible but inconsistent heuristics as well. UCS assumes nonnegative costs; the A* optimality claim also assumes an admissible heuristic with zero at goals.

Table 2. Weighted position search. StayEast charges 0.5^x ; StayWest charges 2^x for entering column x .

Layout	Search	Steps	Path cost	Expanded	Time (ms)	Peak (KiB)
mediumDottedMaze	BFS	68	1.000982522848	208	0.204	41.4
mediumDottedMaze	UCS	74	1.000976583804	186	0.206	46.2
mediumScaryMaze	BFS	72	70,670,371,838	279	0.296	46.4
mediumScaryMaze	UCS	92	68,719,479,864	108	0.136	31.4

The longer UCS routes are cheaper under the weighted objectives. On mediumDottedMaze, 74 moves cost about 1.000976584, slightly below BFS's 68-move cost of 1.000982523. On mediumScaryMaze, UCS takes 92 moves but reduces cost from 70,670,371,838 to 68,719,479,864. Counting moves alone would select the wrong objective. The supplied agent console rounds costs to integers; the benchmark records the full values.

What Manhattan distance contributes

For unit-cost four-direction movement, Manhattan distance ignores walls. It cannot exceed the remaining maze distance, is zero at the goal, and changes by at most one per move. It is therefore admissible and consistent for the unit-cost position problems used here. It is not assumed valid without rescaling for the exponential weighted objectives.

On **bigMaze**, Manhattan A* expands **549** states versus UCS's **620**, an 11.5% reduction, with the same 210-step path. The corresponding medians are 0.628 and 0.687 ms. These counts match the assignment's reference counts.

On **openMaze**, DFS takes 298 moves; BFS, UCS, and A* each take 54. A* expands 535 states versus 682 for BFS/UCS. Even in an open region, many states can share the same **g + h** value; insertion-order tie-breaking still explores a broad plateau. A correct heuristic does not imply that A* visits only the final route.

Time depends on more than expansions

BFS and UCS expand the same unit-cost states, but priority-queue management adds overhead to UCS. Heuristic evaluation also has a cost: on tinyMaze, saving one expansion produces no decisive speed advantage. Compare cost quality, expansions, time, and memory together.

Q5 AND Q6

5. Visiting all four corners

The abstract state is $((x, y), \text{visited_mask})$. Bit i records whether corner i has been visited. The four corners are ordered bottom-left, top-left, bottom-right, top-right. For example, binary 0101 records the two bottom corners. The goal mask is 1111 (15).

A start position on a corner sets that bit immediately. Every legal successor moves one square, copies the integer mask, sets any newly visited corner bit, and costs one. No ghost positions or food grid enter the state. Tuples and integer masks prevent one search branch from mutating another branch's history. With V walkable cells, there are at most $16V$ such states, though many combinations are unreachable.

Heuristic definition

```
h_C(p, R) = min over orders (c1, ..., ck) of R:
  Manhattan(p, c1) + sum Manhattan(ci, ci+1)
h_C(p, empty) = 0
```

R is the set of unvisited corners. The heuristic selects the cheapest order after removing all walls. There are at most $4! = 24$ orders. Corner-to-corner suffix costs are cached by visit mask; only the distance from the current position changes from state to state. This solves a small relaxed problem, not the original maze problem.

Why the heuristic is admissible and consistent

Admissibility. Any real completion induces an order of first visits to the remaining corners. Each segment is at least its Manhattan distance. Minimizing these segment sums over all orders is therefore a lower bound on every real completion.

Consistency. Removing walls keeps every real move legal. One legal unit move followed by an optimal relaxed completion of its successor is a relaxed completion of the current state. Thus $h(s) \leq 1 + h(s')$. The estimate is nonnegative and equals zero at every goal.

Table 3. Corners search; optimal lengths independently matched UCS in the test suite.

Layout	Search	Steps / cost	Expanded	Time (ms)	Peak (KiB)
tinyCorners	BFS	28	252	0.319	51.4
tinyCorners	A*	28	159	0.391	56.9
mediumCorners	BFS	106	1,966	2.477	417.6
mediumCorners	A*	106	741	1.724	196.8

On **mediumCorners**, A* reduces expansions from 1,966 to 741 (62.3%) and peak traced memory from 417.6 to 196.8 KiB. Its 1.724 ms median is below BFS's 2.477 ms. The 741 expansions satisfy the full-credit threshold of fewer than 1,200 and the assignment's stronger fewer-than-800 target.

On **tinyCorners**, both methods find 28 steps, but A* is slower (0.391 versus 0.319 ms) and uses slightly more memory. The small problem does not save enough search work to repay the heuristic and cache overhead.

Q7

6. Collecting all food

The supplied food state is `(position, foodGrid)`. Let F be the remaining food coordinates and $d(a, b)$ be shortest-path distance through the fixed maze. The heuristic is:

```
h_F(p, F) = min d(p, food) + MST_d(F), for nonempty F
h_F(p, empty) = 0
```

$MST_d(F)$ is the weight of a minimum spanning tree on food locations using maze distances as edge weights. Breadth-first distance maps are cached once per food location; Prim's algorithm computes tree weights, cached by food subset. Neither preprocessing step modifies a search state or solves the food-collection problem.

Correctness argument

Admissibility. Any completion first reaches some food at cost at least the nearest-food distance. The remaining walk connects all foods, so its cost is at least the weight of a minimum spanning tree. Adding these two lower bounds is valid because they bound consecutive, nonoverlapping portions of the walk.

Consistency without eating. The food set and MST are unchanged. A unit move can reduce the nearest-food maze distance by at most one, so $h(s) \leq 1 + h(s')$.

Consistency when eating q . Let $F' = F - \{q\}$. Before the move the nearest-food distance is at most one. If F' is nonempty, attaching q to its nearest node in an MST of F' makes a valid spanning tree of F . Therefore $MST(F) \leq MST(F') + \min d(q, F')$, which gives $h(s) \leq 1 + h(s')$. When q is the final food, the previous estimate is at most one and the successor estimate is zero. Nonnegativity and the goal condition also hold.

Table 4. Food search. A* uses the MST heuristic; UCS is the null-heuristic baseline.

Layout	Search	Steps / cost	Expanded	Time (ms)	Peak (KiB)
testSearch	UCS	7	14	0.079	16.0
testSearch	A*	7	7	0.063	12.2
tinySearch	UCS	27	5,057	62.635	8,238.0
tinySearch	A*	27	89	1.695	243.1
trickySearch	UCS	60	16,688	456.943	47,894.3
trickySearch	A*	60	255	9.188	1,026.8

trickySearch: 255 expansions are below every listed threshold, including the fewer-than-7,000 extra-credit criterion. Compared with UCS, the measured search is 49.7 times faster and uses 97.9% less peak traced memory. Both routes cost 60. On tinySearch, both cost 27; A* reduces expansions from 5,057 to 89.

Optional mediumSearch challenge. This layout has 108 food dots. A separate five-second diagnostic with memory tracing did not return a solution: 2,072 states expanded, 20,125.1 KiB peak, 5.008 seconds including tracing and interruption cleanup. This is a bounded partial run, not a completion time or an optimality result. Its timings are not comparable with the untraced table.

VALIDATION AND CONCLUSION

7. Evidence, limitations, and analysis

13 local regression tests and 20 complete-game examples passed on Python 3.11. Returned actions were replayed to check legal moves and goal completion. Benchmark path length, cost, and expansion count agreed across all six runs per case.

Validation	Coverage and observed result
Generic search	60 seeded weighted graphs checked against independent optimal-cost references, including BFS and A* with random admissible estimates.
Edge cases	Start at goal; unreachable goals; cycles; zero-cost edges; stale entries; goal-on-pop; DFS order; inconsistent-heuristic reopening; eight-puzzle states.
Heuristic checks	11,540 states and 25,408 transitions across tinyCorners, mediumCorners, testSearch, and tinySearch. Reverse BFS gives exact remaining costs. All admissibility, consistency, nonnegativity, and goal-zero checks passed.
Problem checks	Independent corner histories; corner at start; unchanged food grids; optimal costs matched UCS; heuristic expansion targets passed.

Limits of the evidence

The archive has test support classes but no official `autograder.py` or test cases. These are local results, not an official grade. Exhaustive checks cover the listed mazes; the proofs justify the heuristics more generally. Headless runs validate game execution, not the graphical window.

The eight-puzzle test initializes the starter's module-global puzzle as its main block does. Food states grow exponentially with dot count, and caches grow with encountered subsets.

Reproduction

```
python -m unittest discover -s validation -p 'test_*.py' -v
python validation/run_examples.py
python validation/benchmark.py
```

Run from the extracted project directory using Python 3.11. The validation folder includes scripts, logs, CSV statistics, JSON timing samples, and the optional mediumSearch probe.

Critical conclusion

DFS quickly finds feasible routes, but its 130 versus 68 moves on mediumMaze and 298 versus 54 on openMaze show its path-quality risk. **BFS** is a strong equal-cost baseline. **UCS** handles weighted objectives correctly, as the longer but cheaper east/west routes demonstrate.

A* benefits most when a cheap lower bound captures substantial remaining work. Manhattan distance offers modest savings; the corner tour and food MST represent multiple outstanding goals and cut search more sharply. The tinyCorners slowdown shows why fewer expansions alone are insufficient. Cost-optimality, time, memory, and heuristic overhead together favor A* on the harder required problems; the 108-food challenge remains unresolved.